

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: MLA0303

COURSE NAME: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO5: *Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)*

Assessment Tool Weightage: 30%

Dr DUMPALA PRASANTH

QUESTIONS: 10

Total Marks: 50

Annexure A

Analytical Problem-Solving Assessment

Duration: 1hr

1. A warehouse robot must determine the shortest path to transport goods while avoiding obstacles. Apply **Dynamic Programming** to develop an optimal policy and explain how value iteration can improve navigation efficiency. **(5 Marks)**

2. A recommendation system collects customer interactions over multiple sessions. Apply **Monte Carlo Control** to update the policy and explain how the agent improves recommendations with experience. **(5 Marks)**

3. A delivery robot operates in an uncertain environment. Apply the **SARSA** algorithm to update the action-value function and explain how the learned policy adapts to environmental changes. **(5 Marks)**

4. A robotic arm performs continuous object manipulation. Apply the **REINFORCE** algorithm to develop a policy that maximizes task completion accuracy. **(5 Marks)**

5. An autonomous vehicle is trained using **Q-Learning** and **Deep Q-Networks (DQN)**. Analyze the differences in learning performance, convergence speed, and scalability for complex driving environments. **(5 Marks)**

6. A game-playing AI is implemented using **DQN**, **Double DQN (DDQN)**, and **Dueling DQN**. Analyze the advantages of DDQN and Dueling DQN over the standard DQN architecture. . (5 Marks)

7. A smart drone navigation system uses **A2C** and **A3C** algorithms. Analyze the impact of parallel learning on convergence speed, computational efficiency, and policy stability. . (5 Marks)

8. A healthcare decision-support system operates with incomplete patient information. Analyze how a Partially Observable Markov Decision Process (POMDP) improves decision-making compared to a standard MDP. . (5 Marks)

9. A robotic navigation system is trained using PPO, TRPO, and DDPG. Evaluate the suitability of each algorithm for continuous control tasks based on stability, sample efficiency, and computational complexity. . (5 Marks)

10. A smart city traffic management system uses **Multi-Agent Reinforcement Learning (MARL)** to coordinate traffic signals. Evaluate the **effectiveness of MARL** in improving traffic flow while considering scalability, coordination, fairness, and ethical challenges. . (5 Marks)

Code of Conduct:

I K Uday Kiran Reddy (192425211) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:Kuday

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	25	Demonstrates deep understanding of concepts with complete clarity	Explains concepts correctly with minor gaps	Partial understanding with errors or omissions	Unable to explain basic concepts
Application	25	Effectively applies concepts to new situations; solves problems logically	Applies concepts with minor errors	Limited ability to apply concepts; requires guidance	Unable to apply concepts effectively
Analytical Thinking	20	Critically analyzes scenarios with strong justification and reasoning	Provides reasonable analysis with some justification	Superficial analysis with weak reasoning	No analytical reasoning demonstrated
Communication	15	Communicates ideas clearly, confidently, and with appropriate terminology	Communicates clearly but with minor hesitation	Communication lacks clarity or structure	Unable to communicate ideas effectively
Response to Questions	15	Responds accurately and confidently, extending answers with depth	Responds correctly but with limited depth	Responds partially; requires prompting	Unable to respond to questions effectively

1. Warehouse Robot – Dynamic Programming and Value Iteration

Introduction

A warehouse robot needs to move from a starting position to a target location while avoiding obstacles and minimizing the number of movements. This can be modeled as a **Markov Decision Process (MDP)** where the robot's position represents the state and movements such as up, down, left, and right represent actions.

Application of Dynamic Programming

Dynamic Programming (DP) can be used when the environment's transition probabilities and rewards are known.

The **Bellman optimality equation** is:

$$V^*(s) = \max_a \left[R(s, a) + \gamma \sum_{s'} P(s', | sa) V^*(s') \right]$$

Where:

- $V^*(s)$ = optimal value of state s
- $R(s, a)$ = reward for taking action a
- $P(s', | sa)$ = probability of reaching state s'
- γ = discount factor

For example:

- Reaching the destination $\rightarrow$ **+100 reward**
- Hitting an obstacle $\rightarrow$ **-100 reward**
- Each movement $\rightarrow$ **-1 reward**

Value Iteration

Value iteration repeatedly updates the value of every state:

$$V_{k+1}(s) = \max_a \left[R(s, a) + \gamma \sum_{s'} P(s', | sa) V_k(s') \right]$$

The process continues until:

$$| V_{k+1}(s) - V_k(s) | < \epsilon$$

After convergence, the optimal policy is obtained as:

$$\pi^*(s) = \arg \max_a \left[R(s, a) + \gamma \sum_{s'} P(s', | sa) V^*(s') \right]$$

Navigation Efficiency

Value iteration improves navigation because the robot learns which states have higher long-term rewards. It therefore avoids obstacles and unnecessary movements.

Conclusion

Dynamic Programming provides an optimal policy when the warehouse environment is known, while value iteration efficiently computes the shortest and safest path.

2. Recommendation System – Monte Carlo Control

Introduction

A recommendation system interacts with customers over several sessions. The system can use **Monte Carlo Control** to learn which recommendations produce the highest long-term rewards.

The states can represent:

- Customer preferences
- Previous interactions
- Current session information

Actions represent recommended products, movies, recipes, etc.

Rewards can be based on:

- Click $\rightarrow +1$
- Like $\rightarrow +2$
- Purchase $\rightarrow +5$
- Skip $\rightarrow 0$

Monte Carlo Approach

Monte Carlo Control learns action values from **complete episodes** rather than requiring a known transition model.

For an episode:

$$S_0, A_0, R_1, S_1, A_1, \dots, S_T$$

The return from time t is:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

The action-value function is updated using:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[G_t - Q(S_t, A_t)]$$

The policy is then improved using:

$$\pi(S) = \arg \max_A Q(S, A)$$

Improvement Through Experience

Suppose a customer repeatedly clicks on healthy recipes. The recommendation agent receives positive rewards for recommending such recipes. Over many sessions, the corresponding Q -values increase.

The system gradually learns:

Customer interaction → Reward → Update Q-value → Improve policy → Better recommendation

Advantages

- Does not require a model of customer behavior.
- Learns directly from real experiences.
- Can optimize long-term customer engagement.
- Improves recommendations as more sessions are collected.

Conclusion

Monte Carlo Control allows the recommendation system to learn from complete customer experiences and gradually develop a better personalized recommendation policy.

3. Delivery Robot – SARSA

Introduction

A delivery robot operates in an uncertain environment where obstacles, traffic, and movement conditions may change. **SARSA** is an on-policy temporal-difference reinforcement learning algorithm suitable for such environments.

SARSA stands for:

State → Action → Reward → State → Action

SARSA Update

The SARSA update rule is:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma Q(s', a') - Q(s, a)]$$

Where:

- s = current state
- a = current action

- r = reward
- s' = next state
- a' = next action
- α = learning rate
- γ = discount factor

Example

Suppose the robot chooses:

Current location → Move Right → Reward -1 → New location → Move Up

SARSA uses the value of the **actual next action** selected by the current policy.

If the robot reaches the destination:

$$r = +100$$

If it encounters an obstacle:

$$r = -100$$

Adaptation to Environmental Changes

If an obstacle suddenly appears, the robot receives a negative reward. SARSA updates the affected state-action values.

Repeated experience causes the robot to prefer alternative paths.

Thus:

Environment change → New experience → Reward → Q-value update → Policy adaptation

Why SARSA is Suitable

SARSA is an **on-policy** algorithm, meaning that it learns the value of the policy it actually follows. This makes it useful when exploration behavior itself affects safety.

Conclusion

SARSA enables the delivery robot to continuously learn from its interactions and adapt its route when environmental conditions change.

4. Robotic Arm – REINFORCE

Introduction

A robotic arm performs continuous manipulation tasks such as picking, placing, rotating, and assembling objects. The **REINFORCE algorithm** is a policy-gradient method that directly learns a policy.

Instead of learning only $Q(s, a)$, REINFORCE learns policy parameters θ :

$$\pi_{\theta}(a | s)$$

Policy Gradient

The objective is to maximize expected return:

$$J(\theta) = E_{\pi_{\theta}}[G_t]$$

The REINFORCE gradient is:

$$\nabla_{\theta} J(\theta) = E[\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G_t]$$

The policy parameters are updated using:

$$\theta \leftarrow \theta + \alpha G_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

Application

For a robotic arm:

State: Joint angles, object position, velocity

Action: Movement of robotic joints

Reward: Successful object manipulation

Example rewards:

- Correct grasp $\rightarrow +10$
- Successful placement $\rightarrow +50$
- Dropping object $\rightarrow -20$
- Collision $\rightarrow -50$

The robot performs an entire episode, calculates the return, and updates the policy based on the actions that contributed to successful task completion.

Advantages

- Directly optimizes the policy.
- Suitable for continuous action spaces.
- Can learn stochastic policies.
- Does not require a complete environment model.

Limitation

REINFORCE can have **high variance** because updates are based on complete episode returns. Training may therefore be slower than actor-critic methods.

Conclusion

REINFORCE can develop a policy that improves robotic manipulation accuracy by increasing the probability of actions that lead to successful task completion.

5. Autonomous Vehicle – Q-Learning vs DQN

Introduction

Both Q-Learning and Deep Q-Networks learn action values, but they differ significantly in scalability.

Q-Learning

The update equation is:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

Q-Learning generally stores Q -values in a table.

DQN

DQN uses a neural network to approximate:

$$Q(s, a, \theta)$$

Instead of maintaining a table, the neural network can process high-dimensional information such as camera images and sensor readings.

Comparison

Parameter	Q-Learning	DQN
Representation	Q-table	Neural network
State space	Small/discrete	Large/high-dimensional
Image input	Difficult	Suitable
Scalability	Low	High
Memory	Large Q-table for large states	Network parameters
Convergence	Fast for small problems	Requires more training

Parameter	Q-Learning	DQN
Complex driving	Limited	More suitable

Learning Performance

Q-Learning can converge quickly in small environments because the state-action space is manageable. However, autonomous driving contains enormous state spaces.

DQN can generalize across similar states and therefore scales better.

Conclusion

For a simple driving simulator, Q-Learning may be sufficient. For complex autonomous driving environments involving images, sensors, and numerous states, **DQN is more scalable and practical.**

6. DQN vs DDQN vs Dueling DQN

Standard DQN

DQN uses a neural network to estimate:

$$Q(s, a)$$

It uses experience replay and a target network to stabilize training.

However, standard DQN can **overestimate action values** because the same network is used to select and evaluate the maximum-valued action.

Double DQN

DDQN separates action selection from action evaluation.

The target is approximately:

$$y = r + \gamma Q_{\theta^-} \left(s', \arg \max_{a'} Q_{\theta}(s', a') \right)$$

The online network selects the action, while the target network evaluates it.

Advantage of DDQN

- Reduces overestimation bias.
- Produces more accurate value estimates.
- Often provides more stable learning.

Dueling DQN

Dueling DQN separates the network into:

1. **State-value stream**
2. **Advantage stream**

It estimates:

$$Q(s, a) = V(s) + A(s, a)$$

More commonly, the combination is normalized to avoid identifiability issues.

Advantage

Dueling architecture can identify whether a state is valuable even when different actions have similar effects.

Comparison

Feature	DQN	DDQN	Dueling DQN
Overestimation	High	Reduced	Not primary purpose
Value estimation	Standard	More accurate	Separates value and advantage
Stability	Moderate	Better	Better in many environments
Complex states	Good	Good	Very good
Main benefit	Basic deep RL	Reduced overestimation	Better state/action representation

Conclusion

DDQN mainly improves **value estimation**, while Dueling DQN improves **network representation** by separately estimating state value and action advantage. Both can outperform standard DQN in appropriate environments.

7. Smart Drone – A2C vs A3C

Introduction

A drone navigation system needs fast learning because it operates in dynamic environments.

A2C (Advantage Actor-Critic) and **A3C (Asynchronous Advantage Actor-Critic)** combine policy learning with value estimation.

A2C

A2C uses two components:

Actor: Determines which action to take.

Critic: Estimates the value of the current state.

The advantage is:

$$A(s, a) = Q(s, a) - V(s)$$

A common practical estimate is:

$$A_t = R_t - V(s_t)$$

The actor updates the policy using the advantage, while the critic learns the value function.

A3C

A3C uses multiple worker agents that interact with separate environments simultaneously.

Each worker:

1. Collects experience.
2. Computes gradients.
3. Updates shared/global parameters asynchronously.

Comparison

Factor	A2C	A3C
Learning	Synchronous	Asynchronous
Workers	Multiple synchronized workers	Multiple independent workers
Convergence	Stable	Often fast
Hardware use	Efficient with parallel processing	High parallelism
Policy updates	Coordinated	Asynchronous
Complexity	Lower	Higher

Impact of Parallel Learning

Parallel environments generate diverse experiences. This reduces correlation between consecutive experiences and can improve learning speed.

However, A3C's asynchronous updates can introduce instability, while A2C provides more predictable synchronized updates.

Conclusion

A3C can provide fast exploration through asynchronous workers, while A2C offers simpler and often more stable parallel training. For real-time drone navigation, the choice depends on computational resources and stability requirements.

8. Healthcare Decision Support – POMDP

Introduction

A standard **Markov Decision Process (MDP)** assumes that the current state is fully observable. Healthcare systems often do not satisfy this assumption because patient information can be incomplete or uncertain.

A **Partially Observable Markov Decision Process (POMDP)** addresses this problem.

MDP

An MDP consists of:

$$(S, A, P, R, \gamma)$$

where:

- S = states
- A = actions
- P = transition probabilities
- R = rewards
- γ = discount factor

POMDP

A POMDP additionally includes observations:

$$(S, A, P, R, \Omega, O, \gamma)$$

where:

- Ω = observation space
- O = observation probability model

The agent does not directly know the true patient state.

Instead, it maintains a **belief state**:

$$b(s) = P(s \mid \text{observations})$$

Example

A patient may have an unknown disease condition.

Available observations:

- Symptoms
- Blood pressure
- Lab results
- Previous medical history

The system uses these observations to estimate the probability of different underlying conditions.

It can then select actions such as:

- Request additional test.
- Recommend treatment.
- Monitor patient.
- Refer to specialist.

MDP vs POMDP

Feature	MDP	POMDP
State visibility	Fully observable	Partially observable
Uncertainty	Lower	Higher
Observations	State directly available	Indirect observations
Belief state	Not required	Required

Healthcare suitability Limited for incomplete data Highly suitable

Conclusion

POMDP provides better decision-making under uncertainty because it explicitly represents incomplete observations and maintains beliefs about possible patient states.

9. PPO vs TRPO vs DDPG

Introduction

Robotic navigation often involves **continuous control**, such as steering angle, velocity, and motor torque. PPO, TRPO, and DDPG are important algorithms for such tasks.

PPO – Proximal Policy Optimization

PPO restricts excessively large policy updates using a clipped objective.

A simplified objective is:

$$L^{CLIP}(\theta) = E[\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)]$$

where:

$$r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)}$$

Advantages:

- Stable.
- Relatively simple.
- Good general-purpose policy optimization.
- Works well with continuous actions.

TRPO – Trust Region Policy Optimization

TRPO limits the size of policy updates using a trust-region constraint.

Conceptually:

$$\max_{\theta} E[r_t(\theta)A_t]$$

subject to:

$$D_{KL}(\pi_{old} || \pi_{\theta}) \leq \delta$$

Advantages:

- Strong policy stability.
- Prevents destructive updates.

Disadvantage:

- Computationally more complex than PPO.

DDPG – Deep Deterministic Policy Gradient

DDPG uses:

- Actor network
- Critic network
- Experience replay
- Target networks

It produces deterministic continuous actions.

Advantages:

- Designed specifically for continuous control.
- Can be sample-efficient through replay.

Limitations:

- Sensitive to hyperparameters.
- Training can become unstable.
- Exploration requires additional noise.

Comparison

Factor	PPO	TRPO	DDPG
Continuous control	Excellent	Excellent	Excellent
Stability	High	Very high	Moderate
Sample efficiency	Good	Good	Good
Complexity	Moderate	High	Moderate
Policy type	Stochastic	Stochastic	Deterministic
Implementation	Relatively easy	Difficult	Moderate

Evaluation

For a robotic navigation system, **PPO is often the practical choice** because it provides a good balance between stability, performance, and implementation complexity.

TRPO is useful when strict policy-update stability is important. DDPG is attractive for deterministic continuous control but requires careful tuning.

10. Smart City Traffic Management – MARL

Introduction

A smart city can use **Multi-Agent Reinforcement Learning (MARL)** to coordinate traffic signals at multiple intersections.

Each traffic signal acts as an agent.

State

Each agent can observe:

- Number of vehicles.
- Queue length.
- Waiting time.
- Traffic density.
- Current signal phase.

Actions

Actions may include:

- Keep green.
- Switch to red.
- Extend green duration.
- Change signal phase.

Reward

A reward can be designed as:

$$R = -(w_1 \times \text{waiting time} + w_2 \times \text{queue length} + w_3 \times \text{congestion})$$

The goal is to minimize congestion and waiting time.

MARL Workflow

Traffic sensors → State collection → Multiple RL agents → Signal decisions → Traffic response → Reward → Policy update

Effectiveness

MARL allows neighboring traffic signals to cooperate rather than making isolated decisions.

For example, if one intersection becomes congested, neighboring agents can adjust their signal timing to improve traffic flow.

Key Challenges

1. Scalability

A large city may contain thousands of intersections. Training all agents jointly can become computationally expensive.

2. Coordination

Independent agents may make decisions that benefit one intersection but negatively affect another. Coordination mechanisms are therefore necessary.

3. Fairness

Optimization should not favor one road or neighborhood continuously while causing excessive delays elsewhere.

4. Ethical Issues

Traffic systems affect emergency vehicles, pedestrians, public transport, and different communities. Decisions should therefore consider safety and equitable service.

5. Communication

Agents may need to exchange information with neighboring intersections. Communication delays or failures can affect performance.

Advantages

- Reduced traffic congestion.
- Lower average waiting time.
- Better coordination between intersections.
- Adaptation to changing traffic patterns.
- Potential reduction in fuel consumption and emissions.

Conclusion

MARL is highly suitable for smart-city traffic management because multiple traffic signals can learn cooperative strategies. However, successful deployment requires careful attention to scalability, communication, safety, fairness, and ethical decision-making.